

Comprendre les tests automatisés du projet — explication complète et pédagogique

Pour qui ce document ? Toi, en partant de zéro. Aucune connaissance préalable des tests n'est supposée : chaque notion technique est expliquée la première fois qu'elle apparaît. Ce document répond à une demande explicite (point 282 de `docs/demandes.md`, 01/09/2026), volontairement reportée à la toute fin de la grande série de chantiers de tests pour pouvoir couvrir l'ensemble d'un coup plutôt que par petits bouts.

Chiffres au 03/09/2026 : 183 tests côté client (20 fichiers), 55 tests côté serveur (6 fichiers), soit **238 tests automatisés** au total.

Table des matières

1. [C'est quoi, un test automatisé ?](#)
 2. [Le vocabulaire de base \(Vitest\)](#)
 3. [Les 5 familles de tests de ce projet — vue d'ensemble](#)
 4. [Famille 1 — Fonctions pures \(le calcul métier isolé\)](#)
 5. [Famille 2 — Middleware \(la sécurité de l'API\)](#)
 6. [Famille 3 — Composants React \(l'affichage\)](#)
 7. [Famille 4 — Contextes React \(l'état partagé de l'appli\)](#)
 8. [Famille 5a — Pages complètes : état/affichage/API](#)
 9. [Famille 5b — Tests d'intégration \(vraie base de données\)](#)
 10. [Les bugs réels que ces tests ont trouvés](#)
 11. [La couverture de code : qu'est-ce que ça mesure vraiment ?](#)
 12. [Lancer les tests toi-même](#)
-

1. C'est quoi, un test automatisé ?

Imagine que tu viens de terminer une fonctionnalité : le calcul du prix d'un lot, par exemple. Tu ouvres l'appli dans le navigateur, tu remplis un formulaire, tu vérifies que le prix affiché est correct. C'est un **test manuel** : toi, humain, qui vérifies avec tes yeux.

Le problème du test manuel, c'est qu'il ne survit pas au temps. Trois semaines plus tard, tu modifies une fonction complètement différente (par exemple, l'export PDF) — et sans le savoir, tu casses le calcul du prix d'un lot, parce que les deux bouts de code partageaient une fonction commune. Personne ne rouvre le formulaire de prix pour vérifier, parce que personne n'a de raison de penser que ce coin-là a bougé. Le bug part en production sans être vu. C'est ce qu'on appelle une **régression** : un comportement qui marchait, et qui casse suite à un changement ailleurs.

Un **test automatisé**, c'est la même vérification que tu ferais à la main — mais écrite une fois sous forme de code, et rejouée automatiquement à chaque modification, en quelques secondes, sans que personne n'ait à y penser. Concrètement, dans ce projet : `npm test` relance en quelques secondes 183 vérifications côté client et 55 côté serveur (238 au total) — l'équivalent de rouvrir chaque page, cliquer chaque bouton, remplir chaque formulaire, vérifier chaque calcul à la main. Impossible à faire un humain à chaque modification, trivial pour un ordinateur.

Ce que ça n'est PAS : un test automatisé ne "devine" jamais si le code est bon. Il vérifie seulement ce qu'on lui a explicitement demandé de vérifier. Un test mal écrit (qui vérifie la mauvaise chose, ou rien du tout) donne une fausse impression de sécurité — c'est pour ça qu'un souci récurrent dans ce document sera : "comment on sait qu'un test vérifie vraiment ce qu'il faut, et pas juste qu'il *pass*e".

2. Le vocabulaire de base (Vitest)

Ce projet utilise **Vitest**, l'outil de test standard pour un projet Vite (le même outil que celui qui fait tourner le serveur de développement du client). Quelques mots que tu vas croiser partout dans ce document et dans le code :

- **describe(...)** : regroupe plusieurs tests qui parlent de la même chose (ex: `describe('calculerStatutAutomatique', ...)` regroupe tous les tests de cette seule fonction). Purement organisationnel, aucun effet sur le résultat.
 - **it(...)** (ou son synonyme **test(...)**) : **un test**, un seul. Son nom est une phrase en français décrivant EXACTEMENT ce qui est vérifié (ex: `it("renvoie 0€ sur un avoir (montant négatif) avec la règle par défaut", ...)`) — dans ce projet, ces noms sont volontairement très explicites, pour qu'on comprenne l'intention sans lire le code.
 - **expect(valeurObtenue).toBe(valeurAttendue)** : une **assertion** — la vérification elle-même. "Je m'attends à ce que X soit égal à Y." Si ce n'est pas le cas, le test échoue et affiche la différence. Variantes fréquentes dans ce projet : `.toEqual( ... )` (compare le contenu d'un objet/tableau, pas juste la référence mémoire), `.toBeInTheDocument()` (l'élément existe bien à l'écran), `.not.toBeInTheDocument()` (l'élément a disparu), `.toHaveBeenCalledWith( ... )` (une fonction simulée a bien été appelée avec tels arguments).
 - **Un "mock"** (verbe *mock*) : remplacer une vraie dépendance (un appel réseau, une fonction d'un autre fichier) par une **version contrôlée par le test**, qui renvoie exactement ce qu'on lui dit de renvoyer. Utilité : on ne veut pas qu'un test dépende d'un vrai serveur allumé, d'une vraie connexion internet, ou du hasard. Exemple central de ce projet : `apiFetch` (la fonction qui fait tous les appels réseau du client) est systématiquement remplacée par un faux `apiFetch` programmé pour répondre "voici la liste des lots" sans jamais toucher un vrai serveur.
 - **Une "fixture"** : les données de test elles-mêmes (un faux lot, un faux programme, une fausse liste d'appels de fonds) — fabriquées à la main dans le fichier de test, pour avoir des données prévisibles et contrôlées plutôt que les vraies données changeantes de la base.
 - **beforeEach(...)** : du code exécuté avant CHAQUE test du fichier — sert le plus souvent à remettre les mocks à zéro, pour qu'un test ne soit jamais influencé par ce qu'a fait le test précédent (piège rencontré concrètement cette session, voir plus loin).
 - **jsdom** : un faux navigateur, simulé en pur JavaScript, sans jamais ouvrir Chrome ou Firefox. Il fournit un DOM (la structure de la page) pour que React puisse "rendre" un composant dans un test, sans qu'aucune fenêtre ne s'ouvre réellement. Beaucoup plus rapide qu'un vrai navigateur, suffisant pour vérifier ce qui s'affiche.
-

3. Les 5 familles de tests de ce projet — vue d'ensemble

Tous les tests de ce projet ne se ressemblent pas — ils testent des niveaux différents de l'application, avec des outils différents. Voici la carte générale, avant le détail fichier par fichier :

Famille	Ce qu'elle teste	Base de données réelle ?	Rapidité	Exemple
1. Fonctions pures	Un calcul isolé (entrée → sortie)	Non	Très rapide	<code>calculerStatutAutomatique(tma)</code>
2. Middleware	La sécurité d'une route Express	Non (faux <code>req / res</code>)	Très rapide	<code>verifierToken</code> , <code>autoriserRoles</code>
3. Composants React	Ce qui s'affiche à l'écran	Non (jsdom)	Rapide	<code>Badge</code> , <code>Bandeau</code> , <code>StatCard</code>
4. Contextes React	L'état partagé entre les pages	Non (jsdom)	Rapide	<code>AuthContext</code> , <code>ProgrammeContext</code>
5a. Pages complètes	État + affichage + appels API d'une page entière	Non (jsdom + API mockée)	Rapide	<code>Lots.render.test.jsx</code>
5b. Intégration serveur	Une vraie séquence de requêtes en base	Oui (base en mémoire)	Plus lente	<code>lots.integration.test.js</code>

La logique générale : plus un test se rapproche du comportement réel de bout en bout (une vraie base de données, un vrai clic sur un vrai bouton), plus il est fiable mais lent et coûteux à écrire — donc réservé aux endroits où une fonction isolée ne suffit pas à garantir que "ça marche vraiment" (typiquement : des fonctions qui font plusieurs allers-retours en base, comme la protection anti-doublon des appels de fonds).

4. Famille 1 — Fonctions pures (le calcul métier isolé)

C'est quoi, une "fonction pure" ?

Une fonction pure, c'est une fonction qui : (1) ne dépend que de ses arguments (jamais d'une variable extérieure imprévisible), et (2) ne modifie rien en dehors d'elle-même (pas d'écriture en base, pas de `fetch`, pas de `console.log`). Elle prend une entrée, renvoie une sortie, un point c'est tout. C'est le type de fonction le plus simple à tester : pas besoin de faux serveur, de fausse base de données, de faux navigateur — juste appeler la fonction avec une entrée connue, et vérifier la sortie.

```
// Exemple simplifié, dans l'esprit de formatteDecimales() (Lots.jsx)
function formatteDecimales(valeur) {
  return new Intl.NumberFormat('fr-FR', { minimumFractionDigits: 2 }).format(valeur)
}
```

Un test de cette fonction n'a besoin de rien d'autre que :

```
expect(formatteDecimales(45)).toBe('45,00').
```

Beaucoup de règles métier de ce projet (VEFA, TMA, appels de fonds) sont volontairement écrites sous cette forme — **extraites** de la page React ou de la route Express qui les utilise, exportées séparément, pour pouvoir les tester isolément. C'est une décision d'architecture explicite du projet (voir `CLAUDE.md` : "logique métier lourde extraite dans `server/utils/*.js`").

Où sont ces tests, et ce qu'ils vérifient

Côté serveur :

- `server/models/Tma.test.js` teste `calculerStatutAutomatique(tma)` et `calculerMontantClient(tma, parametres)` — le cœur du calcul TMA. Point notable : le test "'chiffre' avec montantEntreprises à 0" protège contre un piège JavaScript classique, le **"zéro falsy"** : en JavaScript, `0` est considéré comme une valeur "fausse" par un `if` naïf (`if (montant)` est faux quand `montant` vaut `0`). Or un devis à 0€ (ex: un avoir qui compense exactement une autre ligne) est parfaitement valide et "chiffré" — ce n'est pas la même chose que "pas encore chiffré" (`null`). Le code doit écrire `montant !== null && montant !== undefined`, jamais un simple `if (montant)`. Ce piège du zéro revient dans **au moins 4 fichiers différents** de ce projet (voir section 10) — c'est le bug JavaScript le plus fréquent de toute la base de code.
- `server/routes/lots.test.js` teste `validerDatesCoherentesAvecStatut(lot)` — le garde-fou qui empêche d'enregistrer une date d'étape (option, réservation, acte) sur un lot dont le statut ne l'a pas encore atteinte. C'est le miroir exact de la règle non négociable du

projet : "le statut est toujours déduit des dates, jamais une liste déroulante libre" (`CLAUDE.md`) — ici, on protège la cohérence dans l'autre sens (les dates doivent rester cohérentes avec le statut affiché).

- `server/utils/appelsDeFonds.test.js` teste `calculerEmissionAppel( ... )` — la règle "réglé à l'acte" (si l'acte est signé après que la phase de chantier a été constatée, l'appel est réglé automatiquement à la date de l'acte, pas laissé "en attente"). Utilise `vi.useFakeTimers()` pour contrôler `new Date()` — indispensable dès qu'un calcul dépend explicitement de "la date d'aujourd'hui" : sans figer le temps, le test réussirait un jour et échouerait un autre, au hasard du calendrier. Le bug corrigé dans cette fonction (voir section 10) a été trouvé **en écrivant ce test-là**, pas en observation manuelle.

Côté client :

- `client/src/utils/statuts.test.js` — le fichier le plus dense de cette famille (25 tests). Il teste `formatDate`, `calculerDateLimiteJours`, `calculerDateLimiteMois`, `statutAppel`, `statutPret`, `statutSignature`, `estEntrepriseEnRetard`, `estFactureTmaEnRetard` — c'est-à-dire l'ensemble des statuts "en retard/dans les temps" recalculés uniquement à partir de dates, jamais stockés en base. Encore un piège du zéro falsy testé explicitement (`estEntrepriseEnRetard` avec un devis à 0€).
- `client/src/utils/recherche.test.js` teste `correspondRecherche( ... )` — la barre de recherche générique utilisée sur toutes les pages à tableau. Règle métier : tous les mots tapés doivent se retrouver dans n'importe quel ordre (pas une recherche de phrase exacte), en ignorant la casse, les accents, et les espaces insécables des montants formatés (ex: chercher "5444" doit trouver "5 444,00 €" même si cet espace n'est pas un espace normal — voir l'encadré sur les espaces insécables plus bas).
- `client/src/utils/acquereur.test.js` teste `nomAcquereur(acquereur)` — une petite fonction extraite en 2025 après un audit, car elle était **dupliquée à l'identique dans 3 fichiers** (`Lots.jsx`, `Tma.jsx`, `AppelsDeFonds.jsx`). Ce test protège maintenant un point unique utilisé par les 3 pages, au lieu de 3 copies qui pourraient diverger silencieusement.
- `client/src/utils/codePostal.test.js` teste `chercherCodePostal(commune)` — qui interroge une API publique (`geo.api.gouv.fr`) pour proposer un code postal. Règle métier notable, testée précisément : 1 résultat → code postal proposé automatiquement ; 2 résultats → le premier est retenu comme le plus probable ; 3 résultats ou plus (grande ville à arrondissements) → rien n'est proposé, le champ reste à remplir à la main, "deviner serait plus gênant qu'utile".
- `client/src/utils/export.test.js` teste deux fonctions utilisées par les exports PDF : `nettoyerPourPdf` (voir l'encadré ci-dessous) et `calculerLargeursColonnesFigees` (calcule la largeur de chaque colonne d'un tableau PDF pour qu'il occupe exactement la largeur imprimable de la page, ni plus ni moins).
- `client/src/pages/AppelsDeFonds.test.js`, `Lots.test.js`, `Tma.test.js` testent les fonctions pures exportées en haut de ces 3 grosses pages — à distinguer des fichiers `.render.test.jsx` du même nom (famille 5a, voir plus bas), qui testent autre chose.

`Lots.test.js` est le plus gros (12 fonctions, 25 tests) : formatage des surfaces, pluriel grammaticalement correct des annexes ("Parking extérieur" → "Parkings extérieurs", et pas "Parking extérieurs" — un vrai bug de grammaire française trouvé en écrivant ce test), priorité des dates (acte > réservation > option), détection d'une offre de prêt manquante.

Encadré — le piège des espaces insécables. Quand JavaScript formate un montant à la française (`Intl.NumberFormat('fr-FR')`), le séparateur de milliers n'est pas un espace normal : c'est un caractère spécial invisible (espace insécable, ou espace fine insécable — deux caractères Unicode différents, U+00A0 et U+202F). Ces caractères cassent deux choses indépendamment : la police "helvetica" de jsPDF ne sait pas les afficher (ils apparaissent comme un / dans les PDF générés), et la recherche texte ne les reconnaît pas comme un espace normal (chercher "5444" ne trouvait pas "5 444,00 €"). Deux correctifs distincts existent dans le code (`nettoyerPourPdf` et `recherche.js`), chacun testé séparément — un piège invisible à l'œil nu, documenté aussi dans `CLAUDE.md` .

5. Famille 2 — Middleware (la sécurité de l'API)

`server/middleware/auth.test.js` teste les deux fonctions qui protègent l'intégralité de l'API : `verifierToken` (vérifie le jeton JWT envoyé par le client) et `autoriserRoles` (restreint une route à une liste de rôles). Un **middleware**, en Express, c'est une fonction qui s'exécute *avant* la vraie route, et qui peut soit laisser passer la requête (`next()`), soit la couper court (`res.status( ... ).json( ... )`) — exactement comme un videur à l'entrée d'une salle.

Ce test ne touche ni base de données ni vrai serveur HTTP : il fabrique de faux objets `req / res / next` à la main (via `vi.fn()`), des fonctions "espionnes" qui enregistrent comment elles ont été appelées), et vérifie ce qui se passe. C'est la façon la plus légère de tester du code Express.

Points de sécurité vérifiés concrètement :

- un jeton signé avec un **autre secret** est bien rejeté (empêche de forger un faux jeton admin de toutes pièces) ;
 - un jeton **expiré** (les jetons de ce projet durent 7 jours) est bien rejeté ;
 - un rôle absent de la liste autorisée reçoit bien **403** (interdit) — c'est la barrière technique qui fait respecter les 3 rôles métier (admin / gestionnaire / lecture) ;
 - si une route oublie de préciser des rôles autorisés, `autoriserRoles` **refuse tout le monde par défaut**, plutôt que d'ouvrir par accident — un choix de "sécurité par défaut" volontaire.
-

6. Famille 3 — Composants React (l'affichage)

Un **composant React**, c'est un petit morceau réutilisable d'interface (un bouton, une carte, un bandeau). Tester un composant, c'est le "rendre" dans un faux navigateur (jsdom), puis vérifier ce qui apparaît vraiment à l'écran — texte, classes CSS, éléments présents ou absents.

- **Badge.test.jsx** — le petit pastille colorée de statut (ex: "Refusé", "Validé"). Vérifie que le texte s'affiche et que la bonne classe CSS est appliquée selon le statut (`badge refuse` , `badge valide`) — cette classe détermine la couleur, une erreur de nommage casserait silencieusement l'indication visuelle sans aucune erreur JavaScript pour la signaler.
 - **StatCard.test.jsx** — les cartes de statistiques des tableaux de bord ("Lots au total : 42"). Test notable : *"affiche le bloc pourcentage même à 0"* — encore le piège du zéro falsy, cette fois côté affichage : "0% des lots sont actés" est une information valide (pas une absence de donnée), le composant doit bien l'afficher plutôt que de masquer le bloc entier.
 - **Bandeau.test.jsx** — l'en-tête présent sur toutes les pages protégées, qui combine des informations venant de **deux contextes** différents (utilisateur connecté + programme actif, voir section 7). Vérifie l'affichage du nom/rôle de l'utilisateur, les repères du programme (avec un tiret — si un champ n'est pas renseigné), et que les boutons "Déconnexion"/"Changer de programme" déclenchent bien *deux* actions couplées (vider la session ET naviguer vers la bonne page) — un bug pourrait faire l'une sans l'autre.
 - **RouteProtegee.test.jsx** — le composant qui bloque l'accès à toute page tant qu'aucun utilisateur n'est connecté. Test le plus subtil : *"n'affiche rien pendant la vérification de la session"* — sans cet état intermédiaire, un utilisateur réellement connecté verrait apparaître un "flash" de redirection vers la page de connexion avant que la vérification du jeton (asynchrone, ça prend un instant) ne se termine et ne le renvoie vers sa vraie page.
 - **ChoixProgramme.test.jsx** — la page d'accueil de sélection de programme immobilier. C'est un vrai test d'interaction complet (pas juste de l'affichage) : simuler un clic sur un programme, vérifier qu'on navigue bien vers l'accueil ; vérifier que le formulaire de création n'apparaît que pour les rôles admin/gestionnaire (pas "lecture") ; vérifier qu'un échec de création (nom déjà pris) affiche l'erreur du serveur **sans** naviguer nulle part — sans ce dernier test, un bug pourrait faire croire à l'utilisateur que son programme a été créé alors que non.
-

7. Famille 4 — Contextes React (l'état partagé de l'appli)

Un **contexte React** (`AuthContext` , `ProgrammeContext`), c'est un état partagé par toute l'application sans avoir à le repasser page par page en props. Deux contextes existent dans ce projet :

- **AuthContext** gère qui est connecté. La vraie source de vérité entre deux rechargements de page est le `localStorage` du navigateur (le jeton JWT y est stocké) — le contexte n'en est qu'une **vue réactive** : quand l'état change, React re-rend automatiquement tout ce qui en dépend.
- **ProgrammeContext** gère quel programme immobilier est actif pour la session en cours (ex: "Les Jardins"), pour que toutes les pages (Lots, TMA, Appels de fonds...) filtrent leurs données sur ce programme précis.

`AuthContext.test.jsx` vérifie notamment un point de sécurité important : à chaque chargement de page, un jeton stocké est **revalidé auprès du serveur** (`/api/auth/moi`), jamais fait confiance aveuglément. Si le serveur répond "non" (compte supprimé entre-temps par un admin, jeton périmé), le jeton ET l'utilisateur sont effacés du `localStorage` — empêchant l'appli de laisser croire à une session active qui ne l'est plus.

`ProgrammeContext.test.jsx` contient le test le plus intéressant de toute cette famille : *"ne décide rien tant qu'AuthContext n'a pas fini sa propre vérification"*. C'est le test qui protège un **vrai bug déjà survenu en production** (point 143 de `docs/demandes.md`) : sans ce garde-fou, `ProgrammeContext` se déclenchait une première fois avec `utilisateur` encore à `null` (avant qu' `AuthContext` ait fini de revérifier le jeton auprès du serveur), concluait à tort "personne n'est connecté", et renvoyait l'utilisateur vers l'écran de choix de programme — **à chaque rechargement de page**, même pour quelqu'un de déjà connecté avec un programme déjà choisi. Le test simule ce scénario exact (`chargementAuth: true`) et vérifie que rien ne se décide tant que ce premier chargement n'est pas terminé.

8. Famille 5a — Pages complètes : état/affichage/API

C'est la famille de tests **la plus récente** (chantiers 15 à 17, terminés le 03/09/2026) et la plus proche de "ce qu'un utilisateur fait vraiment". Elle concerne les 3 plus grosses pages du projet (`AppelsDeFonds.jsx` , `Tma.jsx` , `Lots.jsx` — les "god components", des fichiers volumineux qui gèrent à eux seuls le chargement des données, tous les filtres, tous les formulaires d'édition, et tous les exports d'une page entière), dans le but explicite de **sécuriser leur comportement avant de les découper** en morceaux plus petits (point 236, pas encore commencé).

Contrairement aux fonctions pures (famille 1) qui testent UN calcul isolé, ces tests-là font tourner **la page React entière** : chargement des données au montage, clic sur un filtre, ouverture d'un panneau de modification, remplissage d'un formulaire, clic sur "Enregistrer", vérification que le panneau se ferme et que les données affichées sont à jour. `apiFetch` est intégralement simulé (aucun vrai serveur, aucune vraie base de données) — mais tout le reste (le vrai composant React, les vrais formulaires enfants) tourne pour de vrai dans jsdom.

- `AppelsDeFonds.render.test.jsx` (17 tests, la page pilote qui a établi la méthode) — couvre le chargement, les 3 filtres, le récapitulatif par lot, le panneau "Modifier" (ouverture/fermeture exclusive entre les lignes, succès, échec), l'attestation en masse, et les 4 options d'export.
- `Tma.render.test.jsx` (20 tests) — même principe, avec une difficulté technique en plus : ouvrir un panneau déclenche 2 appels réseau supplémentaires (le composant enfant `DetailEntreprisesTma` a son propre chargement). Le mock est passé d'une simple chaîne d'appels dans l'ordre à un **"routeur d'URL"** — une fonction qui regarde l'URL et la méthode de chaque appel simulé pour décider quoi répondre, plus robuste que de compter les appels un par un.
- `Lots.render.test.jsx` (16 tests, dernière page, celle qui vient d'être terminée) — couvre le chargement, l'avertissement "Offre de prêt non reçue", les filtres, le panneau "Modifier" (avec "Annuler la vente"), la modification du prix, la vente d'une annexe seule, l'accordéon Historique (annulations + détail dépliable), les 4 exports. **C'est ce fichier qui a permis de trouver et prouver un vrai bug de production** — détail complet en section 10.

Ces tests suivent tous la même règle : **1 chemin de code = 1 test**, pas la combinatoire de tous les chemins entre eux (ex: on ne teste pas "le filtre statut ET le filtre phase en même temps ET un panneau ouvert" — juste chaque filtre séparément). Décision explicite prise avec Nicolas pour garder une couverture complète sans faire exploser le nombre de tests inutilement.

9. Famille 5b — Tests d'intégration (vraie base de données)

Les tests d'intégration sont les seuls de ce projet à utiliser une **vraie base de données MongoDB** — via `mongodb-memory-server`, qui démarre une instance MongoDB complète mais éphémère, entièrement en mémoire (pas de fichier sur le disque, détruite à la fin des tests). Avant chaque test, la base est vidée (`viderBaseTest`) pour repartir d'un état propre et prévisible.

Pourquoi ne pas se contenter de mocker la base, comme `apiFetch` est mocké côté client ?

Parce que certaines fonctions font *plusieurs* allers-retours en base, avec de vraies requêtes Mongoose (recherche, mise à jour groupée, agrégation entre plusieurs lots ou plusieurs entreprises) — un mock ne suffirait pas à garantir que la vraie requête se comporte comme prévu. La protection anti-doublon en est le meilleur exemple : il faut vraiment interroger la base pour vérifier "cette phase existe-t-elle déjà ?" avant d'en créer une nouvelle.

- `server/routes/lots.integration.test.js` teste 4 fonctions : `genererAppelsDeFonds` (dont le test le plus critique financièrement : "*ne duplique jamais une phase déjà générée*" — sans cette protection, un double-clic créerait des doublons de facturation), `synchroniserAnnexesEtPrix`, `genererAppelsAnnexeSeule`, `resynchroniserMontantReservation` (renégociation de prix avant l'acte, montant figé une fois Acté).
 - `server/routes/tmaEntreprises.integration.test.js` teste `recalculerTma` — le recalcul du statut/montant d'une TMA à chaque ajout/modification/suppression d'un devis entreprise. Test le plus important : "*reste à 'étude' tant qu'une seule entreprise sur deux a répondu*" — protège un bug déjà corrigé (point 136) où la TMA basculait à tort en "chiffré" dès la première réponse, sans attendre les autres entreprises sollicitées.
-

10. Les bugs réels que ces tests ont trouvés

C'est sans doute la partie la plus concrète de ce document : des exemples réels, pas hypothétiques, de bugs trouvés grâce à l'écriture de tests — détail complet de chacun dans `docs/bugs.md`.

1. **Le piège du "zéro falsy"**, rencontré et corrigé **au moins 4 fois** dans des endroits différents du projet (`Tma.js` côté serveur, `statuts.js` côté client, `StatCard.jsx`, et implicitement partout où un montant ou un pourcentage peut légitimement valoir 0). En JavaScript, `0`, `''`, `null`, `undefined` sont tous des valeurs "fausses" pour un `if` — mais `0` est souvent une **vraie donnée valide** (un devis à 0€, 0% du programme), pas une absence de donnée. La correction systématique : comparer explicitement à `null / undefined`, jamais un simple `if (valeur)`.
2. **calculerEmissionAppel** — un `&&` qui renvoie `null` au lieu de `false` (`server/utils/appelsDeFonds.js`). L'expression `lot.dateActe && new Date(lot.dateActe) ≥ new Date(dateAttestation)` renvoie `null` (pas `false`) quand `lot.dateActe` est vide, à cause du court-circuit du `&&` en JavaScript (il renvoie la valeur qui a fait échouer la condition, pas un booléen). Corrigé en enveloppant le tout dans `!!( ... )`. Trouvé **en écrivant le test automatisé**, pas en test manuel.
3. **recalculerTma** — **bascule prématurée du statut** (point 136, `server/routes/tmaEntreprises.js`) : sans comparer au nombre d'entreprises réellement *sollicitées* (`nombreEntreprisesConcernees`), une TMA basculait en "chiffré" dès la première réponse d'entreprise, même si une deuxième entreprise restait à consulter.
4. **ProgrammeContext** — **redirection en boucle au rechargement** (point 143, déjà mentionné en section 7) : décision prise avant qu' `AuthContext` ait fini sa propre vérification.
5. **pluraliser** — **accord grammatical incorrect** (`Lots.jsx`, chantier

13. : un `+s` naïf en fin de libellé composé donnait "Parking extérieurs" au lieu de "Parkings extérieurs" — corrigé pour accorder chaque mot du libellé, pas seulement le dernier.
6. **setIdPrixEnEdition** — **le bug le plus récent** (03/09/2026, `client/src/pages/Lots.jsx`), trouvé en préparant `Lots.render.test.jsx` : la fonction `enregistrerPrixLot()` appelait `setIdPrixEnEdition(null)`, un identifiant **qui n'avait jamais été déclaré nulle part dans le fichier** (code mort d'un ancien refactor). Résultat concret : modifier le prix d'un logement enregistré bien la nouvelle valeur en base (l'appel réseau réussissait), mais le sous-panneau de modification du prix ne se refermait jamais après un succès — comme si "rien ne s'était passé", alors que le prix avait bel et bien changé. **Point technique intéressant** : cette erreur ne plantait PAS l'application (pas de "page

blanche"), parce qu'elle se produisait dans une fonction asynchrone déclenchée par un clic, pas pendant le rendu React — une erreur de ce type devient une "promesse rejetée", invisible à l'écran, qui casse silencieusement uniquement ce qui dépendait de sa suite (ici, la fermeture du panneau). Un test qui vérifie le VRAI comportement observable après une action (pas seulement "l'appel API a été fait") est le seul type de test qui pouvait détecter ce genre de bug.

Le point commun à retenir : aucun de ces bugs n'a été trouvé en « devinant » où chercher — chacun est apparu en écrivant méthodiquement un test pour un comportement qui semblait évident, et en découvrant que le code ne faisait pas *exactement* ce qu'on croyait.

11. La couverture de code : qu'est-ce que ça mesure vraiment ?

La **couverture de code** ("code coverage") est un outil qui répond à une seule question : *"quel pourcentage des lignes de mon code a été exécuté au moins une fois pendant toute la suite de tests ?"* Elle ne dit JAMAIS si le code est correct — juste s'il a été touché du tout. Un fichier à 100% de couverture peut très bien contenir un bug (si le test qui l'exécute ne vérifie pas la bonne chose) ; à l'inverse, un fichier à 0% n'a simplement jamais été testé, ce qui ne veut pas dire qu'il est cassé, juste qu'on n'a aucune garantie automatique dessus.

Dans ce projet, la couverture est mesurée avec `npm run coverage` (`@vitest/coverage-v8`) — un outil de **visibilité**, pas une porte bloquante (pas de seuil minimum imposé qui empêcherait de continuer, voir `docs/specs/tests-automatisees-coverage.md`).

Chiffres du 03/09/2026 (après les chantiers 12 à 17) :

	Couverture globale (instructions)	Repères
Serveur	31,38%	100% sur <code>middleware/auth.js</code> et la plupart des modèles ; <code>lots.js</code> (routes) à 39,74% grâce aux tests d'intégration ; 0% sur <code>index.js</code> / <code>seed.js</code> (points d'entrée, jamais exécutés par les tests) et sur les routes non encore couvertes (<code>programme.js</code> , <code>auth.js</code> , <code>tma.js</code> ...).
Client	44,21%	100% sur tous les <code>utils/</code> , <code>context/</code> , <code>hooks/</code> , <code>data/</code> ; 88 à 94% sur les 3 grosses pages (<code>AppelsDeFonds.jsx</code> , <code>Lots.jsx</code> , <code>Tma.jsx</code>) depuis les chantiers 15-17 ; 0% sur les pages jamais encore testées (<code>Clients.jsx</code> , <code>Connexion.jsx</code> , <code>Parametres.jsx</code> , <code>SuiviPret.jsx</code> , <code>SignatureActe.jsx</code>) et sur la plupart des composants de paramétrage.

Ces chiffres montrent bien la stratégie suivie depuis le début : les fonctions à plus haute valeur métier (calculs, sécurité, les 3 pages principales) d'abord, le reste au fur et à mesure. La progression est réelle : la couverture client est passée de 3,64% (juste après les tout premiers chantiers) à 44,21% aujourd'hui — grâce, en grande partie, aux 3 chantiers de tests de pages complètes de cette dernière semaine.

12. Lancer les tests toi-même

Deux commandes, à exécuter séparément dans chaque dossier (`server/` et `client/`) :

```
# Lance toute la suite de tests une fois, affiche le résultat
npm test

# Lance la suite de tests ET mesure la couverture de code
npm run coverage
```

Un test qui échoue affiche : son nom exact (la phrase en français), la valeur attendue, la valeur réellement obtenue, et souvent (pour les tests de composants React) tout le HTML rendu à l'écran au moment de l'échec — très utile pour comprendre pourquoi un élément qu'on cherchait n'était pas là où on l'attendait.

Dans ce projet, un hook Claude Code (`.claude/hooks/test-on-write.js`) relance automatiquement les tests concernés en arrière-plan à chaque modification d'un fichier de test ou de code source associé — pas besoin de taper la commande à la main à chaque fois pendant une session de travail avec l'assistant (voir `docs/protocole-ia-vefa.md` , étape 7).